

Prac 4:

U25044215 – Hannah Diedrick

U25328986 – Michael Harding

U25260244 – Ethan van Eyden

Task 1:

This project models the organisation and processing of work within a software development repository. A repository holds projects which can contain nested projects or individual issues. Issues represent the actual work developers perform. The system allows work items to be hierarchically organised, traversed in various ways, moved between projects at runtime, progressed through a development lifecycle and dynamically extended with properties like priority levels and tags.

The system addresses the challenge of managing a changing software development work hierarchy without tightly coupling client code to the repository's internal structure. Composite is used for the recursive work hierarchy, Iterator provides controlled traversal, State manages the lifecycle of issues and Decorator allows optional responsibilities to be added to issues dynamically.

<u>Design Pattern</u>	<u>GoF participant</u>	<u>Class</u>
<u>Composite</u>	Component	Component
	Leaf	Issue
	Composite	Composite
	Concrete Composite	Project, Repo
	Client	ProjectRegistry / main()
<u>Iterator</u>	Iterator	ProjectIterator
	ConcreteIterator	DepthFirstIterator
	ConcreteIterator	UnresolvedIssueIterator
	Aggregate	Component / Composite hierarchy
	ConcreteAggregate	Composite, Project, Repo
<u>State</u>	Client	ProjectRegistry / main()
	Context	Issue
	State	State
	ConcreteState	OpenState
	ConcreteState	AssignedState
<u>Decorator</u>	ConcreteState	InProgressState
	ConcreteState	ReviewState
	ConcreteState	ClosedState
	Component	Component
	ConcreteComponent	Issue
	Decorator	IssueDecorator

	ConcreteDecorator	PriorityDecorator
	ConcreteDecorator	TagDecorator
	Client	ProjectRegistry / main()

In our design, inheritance represents the primary “is-a” relationships between classes. Issue, Composite and IssueDecorator all inherit from the common Component abstraction. Project and Repo inherit from Composite while concrete State classes inherit from State and concrete Iterator classes inherit from ProjectIterator. Associations are used where objects interact without owning each other, such as an Iterator traversing Component objects or a Decorator referring to the Component it wraps. Multiplicities indicate how many objects can participate in a relationship; for example, a Composite can contain 0 to any number of child Component objects while a Component can have at most one current State. Ownership is represented separately: ProjectRegistry owns Components using `std::unique_ptr` and Component owns its current State. These are composition relationships shown with filled diamonds. In contrast, Composite only stores non-owning pointers to its children so its relationship to Component is aggregation rather than composition.

The concrete domain classes are Repo, Project and Issue. Repo models the top-level software repository storing information like owner, visibility and programming language. Project represents a group of related work including project-specific details such as a deadline and estimated hours. Since Project inherits from Composite it can contain both Issues and further nested Projects creating a recursive hierarchy. Issue is the leaf work item with a meaningful development lifecycle managed by the State pattern. Runtime responsibilities are handled by PriorityDecorator and TagDecorator which allow issues to gain a priority level or descriptive tag without altering the original Issue class.

The system uses Component as a common abstraction for both individual Issues and grouped work, allowing client code to treat leaves and composites uniformly. Repo and Project specialise Composite, while Issue represents the leaf work item. Projects can contain further Projects, creating a recursive hierarchy without exposing its internal container.

ProjectRegistry is the central owner of work objects, storing them using `std::unique_ptr<Component>`. This gives each Component a single, clearly defined owner and automatically destroys it. Consequently, the Composite hierarchy only stores non-owning raw pointers to its children, avoiding duplicate ownership and preventing the Registry and Composite from attempting to delete the same object.

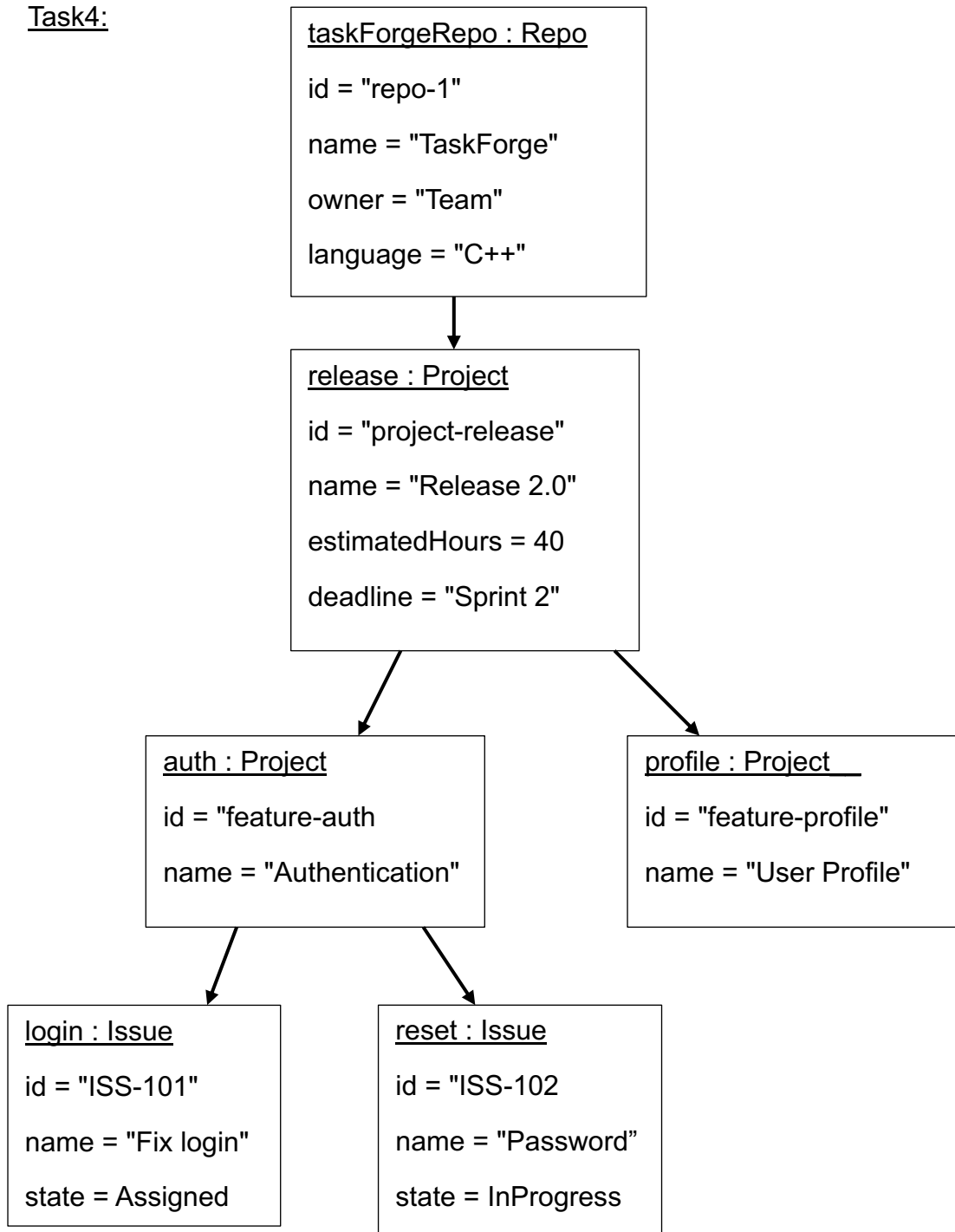
The current lifecycle State is also owned using `std::unique_ptr<State>`. Replacing the State automatically destroys the previous one, while invalid transitions are

rejected by the current concrete State object. This keeps state-dependent logic out of the Issue class and avoids a large centralised conditional structure.

Decorators use non-owning links to the wrapped Component, while the Registry retains ownership of both the wrapped Component and the decorator objects. This allows responsibilities like priority and tags to be dynamically stacked while the decorated object remains usable through the Component interface.

Iterators also use non-owning Component pointers. When created, they build a snapshot of the traversal, ensuring that an existing traversal remains safe and predictable if the hierarchy is modified. A newly created iterator reflects the updated hierarchy.

Task4:



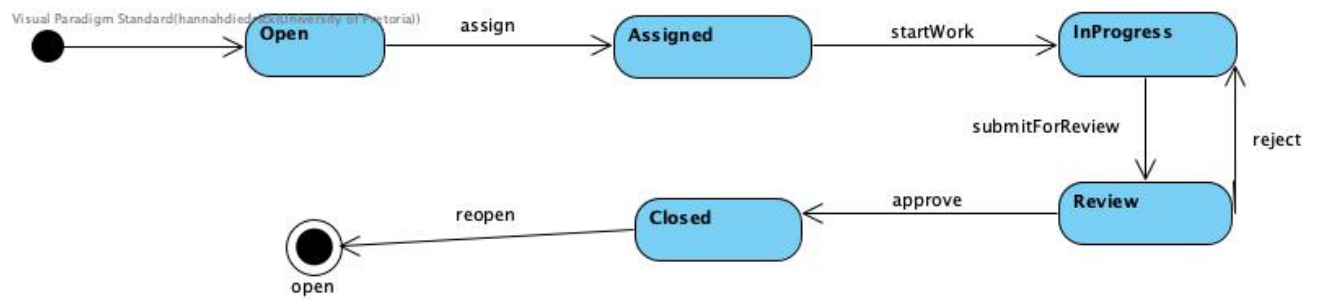


Figure 1: State diagram

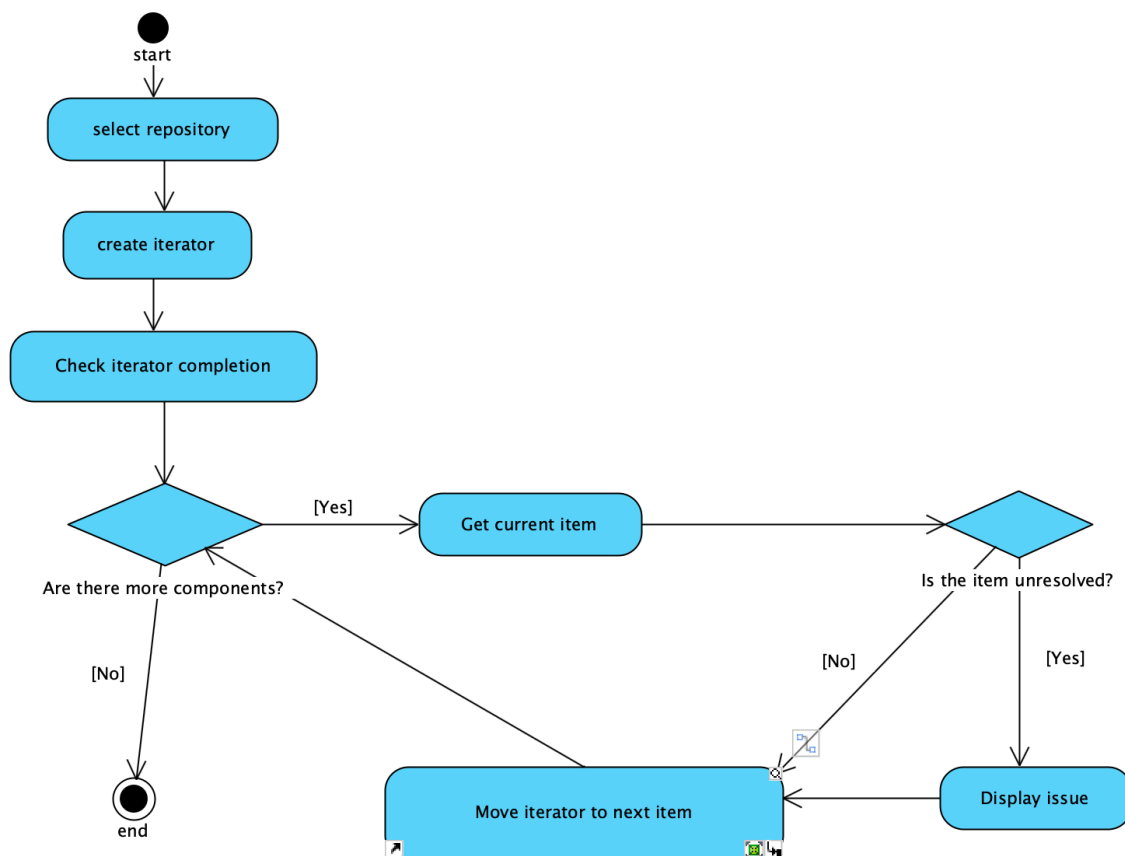


Figure 2: Activity diagram 1

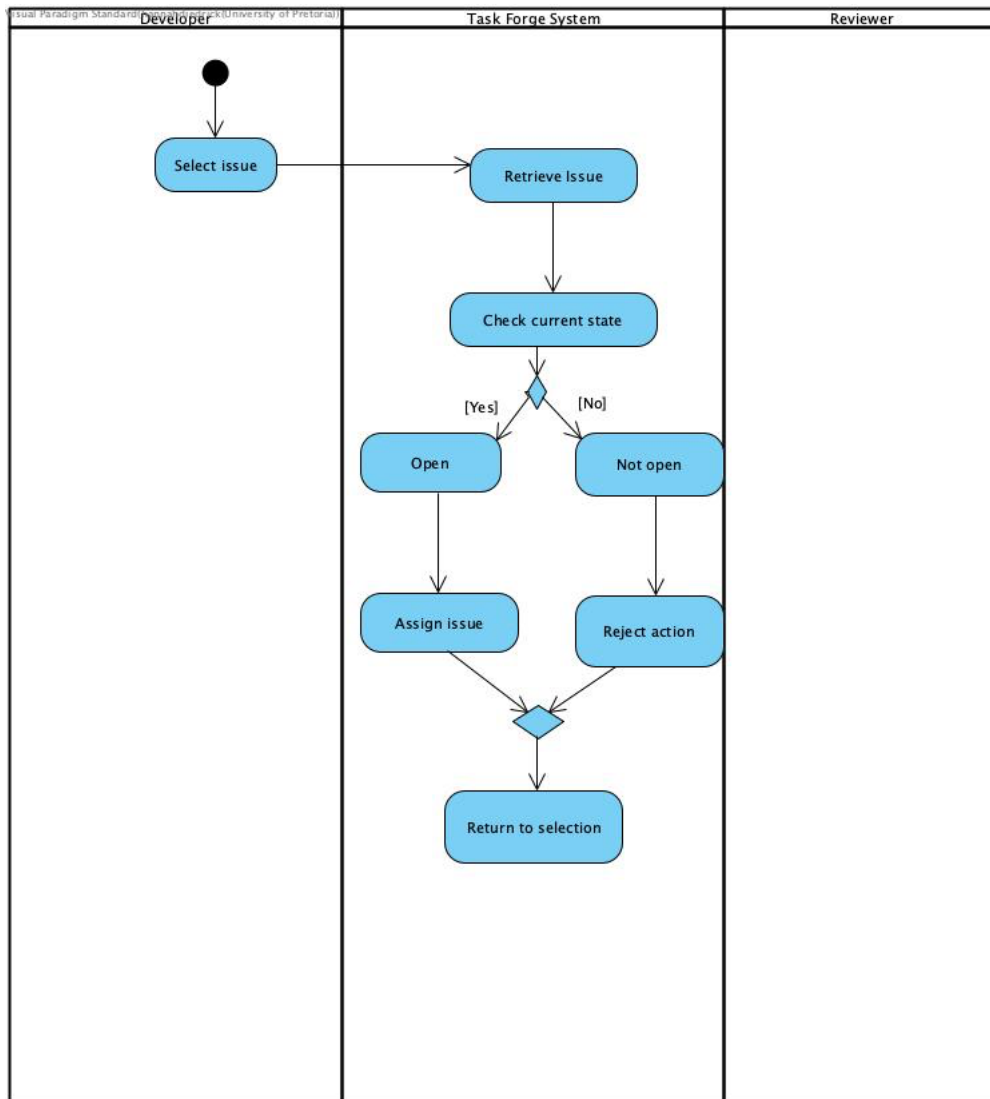


Figure 3:Activity Diagram 2

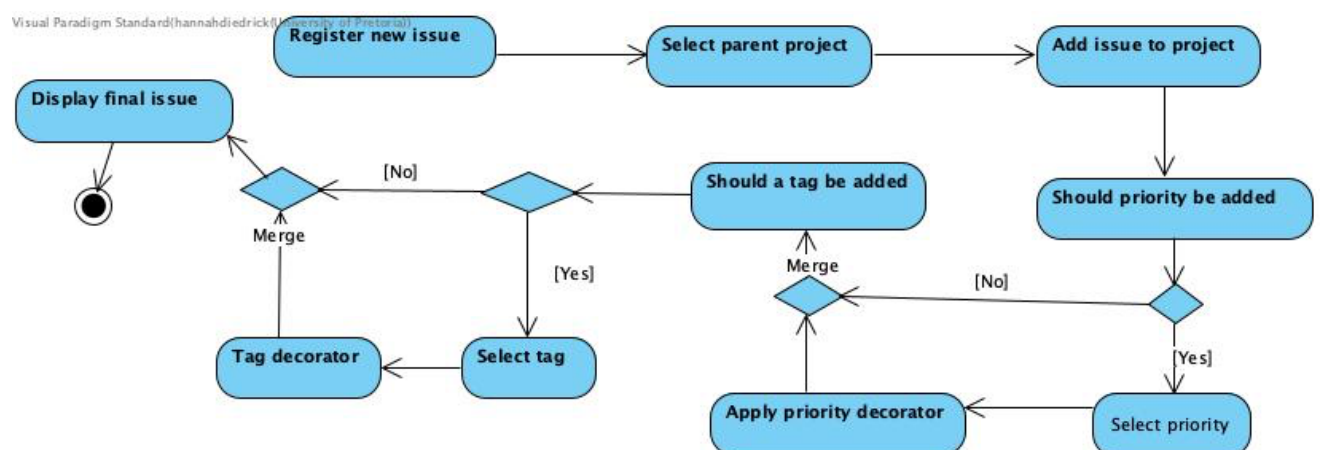


Figure 4: Activity Diagram 3

Task 5:

During the development of TaskForge, GDB was used to inspect runtime behaviour and diagnose pointer-related errors, while Valgrind was used to verify the final program's memory management. The program was compiled with debugging symbols using the `-g` compiler flag so that GDB could inspect source lines, variables and the call stack. A particularly important problem encountered during development involved null pointers in the Decorator implementation. GDB was used to inspect the pointer value and identify the attempted dereference of a null wrapped component. After correcting the error, defensive null checks and regression tests were added. Finally, the complete application was executed under Valgrind to verify that the ownership model did not introduce memory leaks.

During development, a null-pointer error occurred when a Decorator existed without a valid wrapped Component. The Decorator implementation delegated operations such as `getState()`, `getName()` and `getDescription()` to `wrappedComponent`. If `wrappedComponent` was `nullptr`, attempting to call a method through it caused the program to terminate with a segmentation fault.

Figure X: GDB evidence of the null-pointer defect. `wrappedComponent` was inspected and found to contain `0x0`. Execution subsequently attempted to access the wrapped object, causing `SIGSEGV`. The backtrace identified the Decorator implementation as the origin of the failure.

Cause: The Decorator stores a pointer to the Component that it wraps. The original implementation assumed that this pointer would always refer to a valid object and therefore delegated operations to it without first checking its value. When a null pointer was supplied, the Decorator attempted to access an object through address `0x0`. GDB confirmed this by showing that the component/wrapped-component pointer was null immediately before the invalid access.

Correction: Defensive checks were added before every operation that dereferences the wrapped Component. The Decorator constructor also uses conditional expressions when copying the Component's identifying information, allowing construction to remain safe even if a null pointer is supplied. Getter functions return a safe fallback value when no wrapped Component exists, while mutating operations simply do not delegate when the pointer is null. This removed the invalid dereference while maintaining normal Decorator behaviour when a valid Component is supplied.

After correcting the defect, a regression test was added to ensure that the same bug could not silently reappear. The test deliberately constructs `PriorityDecorator` and `TagDecorator` objects with `nullptr` wrapped Components and invokes their normal operations. The assertions verify that these operations return safe values rather than dereferencing the pointer. The test now executes successfully as part of the normal test suite.

Following the null-pointer investigation, similar defensive checks were reviewed throughout the system. Registry operations now verify that requested identifiers exist before dereferencing the returned Component, decoration rejects null Decorators, and Composite operations ignore invalid null children. These checks reduce the possibility of another invalid-memory access entering the system through malformed runtime input.

Valgrind results: The final TaskForge executable was run using `valgrind --leak-check=full --show-leak-kinds=all ./taskforge`. All major runtime scenarios were executed before normal program termination. The final Valgrind report showed 0 bytes definitely lost, confirming that no definite memory leaks originated from the TaskForge implementation. This supports the ownership strategy used by the system, in which owning relationships use `std::unique_ptr` while Composite, Iterator and Decorator navigation relationships use non-owning pointers.

Task 6:

TaskForge development was divided equally among the three team members based on the system's major areas. Ethan focused on implementing the Composite pattern, particularly the hierarchical relationship between repositories projects and issues. He also worked on the Docker environment and contributed to setting up and maintaining the GitHub repository. Michael tackled the Iterator pattern, including the necessary traversal behaviour, and the Decorator pattern, which dynamically adds responsibilities like priority levels and tags to issues. Hannah handled the State pattern, encompassing the issue lifecycle and valid/invalid transitions. She also prepared most of the documentation and UML diagrams needed for submission.

While each member had a primary responsibility, the work was integrated continuously rather than developed as isolated components. The shared Component abstraction enabled the Composite Iterator State and Decorator implementations to interact within the system. Team members regularly merged and tested each other's work, resolving interface and integration issues and ensuring the combined application compiled and functioned correctly. Finally, the main.cpp and demonstration scenarios tested how the four patterns collaborated within the GitHub-style project management domain.

Throughout development, GitHub served as the version control system and a record of collaboration. Team members made commits as their features were implemented and updated, and the repository history documented contributions from all three. Meaningful commits documented changes like implementing pattern classes, fixing integration errors, updating the Docker environment and completing the UML and documentation. Before submission, replace the commit hashes pull requests or issue numbers below with those from the repository.